



INDIAN INSTITUTE OF TECHNOLOGY
KHARAGPUR

Stamp / Signature of the Invigilator

EXAMINATION (Mid Semester)

SEMESTER (Autumn)

Roll Number

Section

Name

Subject Number

C

S

1

0

0

0

3

Subject Name

Programming and Data Structures

Department / Center of the Student

Important Instructions and Guidelines for Students

1. You must occupy your seat as per the Examination Schedule/Sitting Plan.
2. Do not keep mobile phones or any electronic gadgets with you even in the switched off mode. **Calculators are NOT allowed for this exam.**
3. Loose papers, class notes, books or any such materials must not be in your possession, even if they are irrelevant to the subject in which you are taking the examination.
4. Use of pens, pencil, erasers, rulers is allowed during the examination. However, exchange of these items or any other papers (including question papers) is not permitted.
5. You must write your answers in this booklet itself, at the designated places (within the given boxes or blanks). **You can use the indicated pages of the answer script and other white spaces (outside the boxes/blanks) for rough work. No additional sheet will be given.** Do not tear off any page. Report to the invigilator if the answer script has torn or distorted page(s).
6. It is your responsibility to ensure that you have signed the Attendance Sheet. Keep your Admit Card/Identity Card on the desk for checking by the invigilator.
7. You may leave the examination hall for washroom or for drinking water for a very short period. Record your absence from the Examination Hall in the register provided. Smoking and the consumption of any kind of beverages are strictly prohibited inside the Examination Hall.
8. Do not leave the Examination Hall without submitting your answer script to the invigilator. **In any situation, you are not allowed to take the answer script outside the exam hall.** After the completion of the examination, do not leave the seat until the invigilators collect all the answer scripts.
9. During the examination, either inside or outside the Examination Hall, gathering information from any kind of sources or exchanging information with others or any such attempt will be treated as 'unfair means'. Do not adopt unfair means and do not indulge in unseemly behavior.

Violation of any of the above instructions may lead to severe punishment.

Signature of the Student

To be filled in by the examiner

Question Number

1

2

3

4

5

6

7

8

9

10

Total

Marks Obtained

Marks obtained (in words)

Signature of the Examiner

Signature of the Scrutineer

Instructions related to this question paper:

1. This is a QUESTION-CUM-ANSWER script.
 2. **Write your name, roll number and section number (1/2/3/4) carefully on the first page.**
You can know your section number from the name of your PDS Theory teacher:
Dr. Mainack Mondal → you are in section 1
Dr. Pawan Goyal → you are in section 2
Dr. Satrajit Ghosh → you are in section 3
Dr. Saptarshi Ghosh → you are in section 4
 3. Attempt all questions.
 4. **All answers should be written within the boxes provided immediately after / beside the question or as specified in the question.**
 5. **You can use the indicated page(s) of this answer-script and other white spaces (outside the boxes/blanks) for rough work. No additional sheet will be given.** No extra page should be attached with this question-cum-answer script.
 6. No clarifications can be provided during the exam. Make appropriate (but not unjustifiable) assumptions only if necessary, and state your assumptions (if any).
 7. Calculators are NOT allowed during this exam.
-

Space for rough work
Nothing written below on this page will be evaluated

1. (3+3+3 = 9 marks) Consider the following program for parts (a), (b) and (c). Answers are to be written within the answer boxes only.

```
int main()
{
    int n, x, y, r;    char ch;

    // input one int and one char
    scanf("%d %c", &n, &ch);

    x = (n % 5) * 3;
    y = (ch - 'A') + 1;
    r = (x + y * 2) % (n - 3);

    if (r % 2 == 0) r += n / 2;
    else          r -= y;

    printf("x=%d, y=%d, r=%d\n", x, y, r);

    switch (r % 5) {
        case 0:
            printf("Zero\n"); break;
        case 1:
            printf("One\n");
        case 2:
            printf("Two\n");
            r += x / 2; break;
        case 3:
            printf("Three\n");
            r *= 2;
        case 4:
            printf("Four\n");
            r -= y; break;
        default:
            printf("Other\n");
    }
    printf("Final r = %d\n", r);
    return 0;
}
```

The above program is run three times, and the following inputs are given:

- (a) 9 C
- (b) 17 B
- (c) 20 H

Write the outputs of the program for the above inputs.

(a) Output (3 marks):

x = 12, y = 3, r = 4
Four
Final r = 1

(b) Output (3 marks):

x = 6, y = 2, r = 18
Three
Four
Final r = 34

(c) Output (3 marks):

x = 0, y = 8, r = 26
One
Two
Final r = 26

Marks for each part: 3
0.5 for each for values of x, y, r
1 for the output of switch (which
may be one or two lines)
0.5 for the Final r value

2. (2+2+2 = 6 marks) Consider the following code snippet. Note that two statements are commented.

```
int i, j;
for(i=0; i<=5; i++)
{
    for(j=5; j>=i; j--)
    {
        /* if(j%2 == 0) continue; */
        /* if(j <= 3) break; */
        printf("%d", i);
    }
    printf("\n");
}
```

(a) What is the output of the given snippet?

(b) Now, only the statement “if(j%2==0) continue;” is *uncommented*; the next statement still remains commented. What is the output of the modified snippet?

(c) Finally, both the commented statements in the snippet are *uncommented*. What is the output of the modified snippet?

(a) Output (2 marks):

000000
11111
2222
333
44
5

(b) Output (2 marks):

000
111
22
33
4
5

(c) Output (2 marks):

0
1
2
3
4
5

For each part: award 1 mark if at least 50% lines are correct

3. (8 marks) Answer all parts (answers to be written within the answer boxes only).

(a) (3 marks) The function `partition` rearranges an array of integers, so that all negative numbers come before all positive numbers, *in place*, using two indices that move toward each other. Fill in the four blanks and provide the exact output of the program.

```
void partition(int A[], int n){
    int i = 0, j = _____; /*1*/
    while(i <= j){
        if(A[i] < 0)
            _____; /*2*/
        else {
            int t = A[i];
            A[i] = A[j];
            A[j] = ____; /*3*/
            _____; /*4*/
        }
    }
}

int main(){
    int a[] = {-2,4,-1,5,-3};
    int n = 5, i;
    partition(a, n);
    for(i=0;i<n;i++)
        printf("%d ", a[i]);
    return 0;
}
```

Blanks (0.5 each):

(1) `n - 1`

(2) `i++` or equivalent

(3) `t`

(4) `j--` or equivalent

Output (1 mark):

-2 -3 -1 5 4

- (b) (3 marks) The function below returns the *second smallest* element in an integer array, in a single traversal after initialization. Assume there the array contains more than 2 elements, and all elements are distinct. Fill in the three blanks to make the function behave appropriately.

```
int secondMin(int A[], int n){
    int min, second, i;
    min = A[0];
    second = A[1];
    if(_____) /*1*/
    {
        second = A[0];
        min = A[1];
    }
    for(i = 2; i < n; i++){
        if(_____) /*2*/
        {
            second = min;
            min = A[i];
        }
        else if(_____) /*3*/
            second = A[i];
    }
    return second;
}
```

Blanks (1 each):

Equivalent expressions
should be given marks

(1) $A[0] > A[1]$

(2) $A[i] < \text{min}$

(3) $A[i] < \text{second}$

- (c) (2 marks) The function given below tracks the *longest contiguous run of equal elements* in an integer array. Fill in the blank to make the function behave appropriately, and state the output for the given array A.

/ A = {2,2,2,2,3,2,2,2,2,7} */*

```
int longestRun(int A[], int n){
    int i, count = 1;
    int maxcount = 1, maxbegin = 0;
    for(i = 1; i < n; i++){
        if(A[i] == A[i-1])
            count++;
        else
            count = 1;
        if(count >= maxcount){
            maxcount = count;
            maxbegin = ____; /*1*/
        }
    }
    printf("Start = %d, Length = %d,
           Value = %d",
           maxbegin, maxcount,
           A[maxbegin]);
}
```

Blank: (1 mark)

(1) $i - \text{count} + 1$

Output: (1 mark)

Start=5, Length=4, Value=2

4. (7 marks) Answer all parts (answers to be written within the answer boxes only).

(a) (2 marks) What does the program do? Write the exact output of the program.

```
#include<stdio.h>
int main(){
    char s[] = "xyz";
    int i;
    for(i=0; s[i]!='\0'; i++){
        s[i] = 'a'+(s[i]-'a'+3)%26;
    }
    printf("%s", s);
    return 0;
}
```

Computes: (1 mark)

Shifts each letter by 3,
wrapping (z -> c).

Output: (1 mark)

abc

(b) (2 marks) The program tries to check whether two strings are the same. What is the error (if any) in the program? If there is no error, write "No error"; if there is an error, describe the error in the provided box and also provide the *minimum* required change.

```
#include<stdio.h>
int main(){
    char a[] = "hello";
    char b[] = "hello";
    if(a == b)
        printf("Equal");
    else
        printf("Not Equal");
    return 0;
}
```

YES, there is an error.
a==b compares addresses,
not text, so it prints
"Not Equal". Fix:
1. Use strcmp(a,b)==0
instead of a == b.
2. Need #include<string.h>

Marks: 1 for error

1 for fix (0.5 for each)

(c) (3 marks) The function below counts all occurrences of a pattern *pat* in a string *str*. Fill in the three blanks appropriately and mention the output for the string provided in the comment at the end.

```
int countMatches(char str[],
                  char pat[]){
    int n = strlen(str);
    int m = strlen(pat);
    int i, j, count = 0;
    for(i = 0; i <= ____; i++){ /*1*/
        for(j = 0; j < m; j++){
            if(str[i+j] != pat[j])
                break;
        }
        if(____) /*2*/
            count++;
    }
    return ____; /*3*/
}
/* Use str = "aaaaa", pat = "aa" */
```

Blanks:

(1) n - m (1 mark)

(2) j == m (1 mark)

(3) count (0.5 mark)

Returns:

4 (0.5 mark)

5. (2 + 1 + 1 + 1 + 2 + 1 = 8 marks) Answer the following questions (answers to be written within the answer boxes only).

(a) Write the exact output of this program. Spaces matter.

```
#include <stdio.h>
#define SHOW(e) printf(#e "=%d\n", (e))

int sq(int x) { return x * x; }

int noisy(int x) {
    printf("<%d>", x);
    return x;
}

int main(void) {
    int n = printf("IIT\n");
    printf("%d\n", n);
    int r = noisy(0) && noisy(5);
    printf(" %d\n", r);
    SHOW(sq(3) + 1);
    printf("%d\n", printf("ab"));
    return 0;
}
```

Output:

```
IIT
4
<0> 0
sq(3) + 1=10
ab2
```

Marks: 4×0.5 . Unit 1 is the pair IIT and 4; then <0> 0; then the SHOW line; then ab2.

Why: `printf` returns the number of characters printed (IIT\n is 4). `noisy(0)` returns 0, so `&&` is false and `noisy(5)` is never called. `#e` gives the text `sq(3) + 1`. In the last line the inner `printf` runs first (prints ab, returns 2).

(b) Would any of the two lines marked A and B below make the code INVALID (a compile-time error)? Write which line (A or B or None). Justify (in one sentence). No marks without a correct justification.

```
#include <stdio.h>

void f(int n) {
    {
        int n = 5;          /* A */
        printf("%d ", n);
    }
    int n = 7;              /* B */
    printf("%d\n", n);
}

int main(void) {
    f(1);
    return 0;
}
```

Line (A or B or None):

B

Justification:

The parameter `n` already lives in the function's outermost block, so `n` is declared twice there. Line A is inside a new inner block, where shadowing is legal.

Marks: 1.0 = 0.5 for the line + 0.5 for the justification. No marks at all if no or wrong justification is given.

(c) Is the code below VALID, or is it INVALID (gives compile-time error)? Justify in one sentence. No marks without correct justification.

```
#include <stdio.h>

int g(int x) { return x + 1; }

int k = g(3);    /* global */

int main(void) {
    printf("%d\n", k);
    return 0;
}
```

(d) What does this program below print?

```
#include <stdio.h>

int half(double d) { return d / 2; }

double third(int n) { return n / 3; }

int main(void) {
    int a = half(9.0);
    double b = third(10);
    printf("%d %.2f\n", a, b);
    return 0;
}
```

Valid / Invalid:

INVALID

Why:

The initializer of a global variable must be a compile-time constant; a function call is not allowed.

Marks: 1.0 = 0.5 for the verdict + 0.5 for the justification. No marks if no/wrong justification given.

Output:

4 3.00

Marks: 0.5 each for the two values.

Why: the value of a return expression is converted to the function's return type: $9.0/2 = 4.5$ becomes the int 4. In `third`, $10/3$ is an *integer* division (giving 3) *before* the conversion to double, so the result is 3.00, not 3.33.

(e) The code below would read one line typed by the user and after the user presses Enter should print every second character of it, starting from the last one. E.g., for the line IITKGP it prints PKI.

(i) Fill spaces marked A and B with correct code.

(ii) Write the output for the input ABCDE GHIJK\t\t where \t is the tab character.

```
#include <stdio.h>

int rev2(void) {
    int c = getchar();
    if (c == '\n' || c == EOF)
        return 0;
    int after = rev2();
    if ( ____ [A] ____ )
        putchar(c);
    return ____ [B] ____;
}

int main(void) {
    rev2();
    printf("\n");
    return 0;
}
```

(i) Fill spaces:

A (0.5 mark): [See below](#)

B (0.5 mark): [See below](#)

(ii) Output (1 mark):

[TAB] KIGECA

1 for the tab plus KIGECA; 0.5 for KIGECA without the leading tab.

Marks: [A] 0.5, [B] 0.5, (ii) 1. [A] and [B] are marked as a pair.

Correct pairs:

[A]	[B]
after % 2 == 0	after + 1
!(after % 2)	after + 1
after % 2 != 1	after + 1
(after & 1) == 0	after + 1
!(after & 1)	after + 1
after == 0	!after
!after	!after
after == 0	1 - after
after == 0	after ^ 1
after == 0	(after + 1) % 2
after % 2 == 0	!after
after % 2 == 0	after - 1

Notes for identifying correct answers:

- after + 1 may also be written 1 + after, ++after or after += 1 (but *not* after++).
- after == 0 and !after are the same test, and so are after % 2 == 0 and !(after % 2).
- A correct [A] must be *true* when after is 0, because the last character is always printed.
- The parentheses in (after & 1) == 0 are required. Without them it parses as after & (1 == 0), which is always 0.
- Check the pair together: after == 0 is correct only with a flag-style [B] (one that returns 0 or 1), never with after + 1.
- Any pair not listed: the last character always sees after = 0, and each character passes its [B] value to the character before it. So substitute into the [B] written in the answer to get after for the last four characters, e.g. after + 1 gives 0, 1, 2, 3 and !after gives 0, 1, 0, 1. The pair written in answer is correct if [A] is true, false, true, false for these four values. Of course this logic will generalize for cases more than four characters

(ii) The line has 13 characters (the space and the two tabs count). Counting from the end, the printed ones are the second tab, K, I, G, E, C, A. The space is an ordinary character and is skipped. Output: a TAB character followed immediately by KIGECA. Marks: 1 for TAB followed by KIGECA; 0.5 for KIGECA without the leading tab.

(f) X and Y are two complete programs. For each one, write exactly the single line of output that it produces.

```
/* ----- Program X ----- */
#include <stdio.h>

int f(int n) {
    static int calls = 0;
    int mine = ++calls;
    if (n > 0) f(n - 1);
    printf("%d ", mine);
    return calls;
}

int main(void) {
    printf("%d\n", f(3));
    return 0;
}

/* ----- Program Y ----- */
#include <stdio.h>

int f(int n) {
    int calls = 0;
    int mine = calls++;
    if (n > 0) f(n - 1);
    printf("%d ", mine);
    return calls;
}

int main(void) {
    printf("%d\n", f(3));
    return 0;
}
```

Program X prints:

4 3 2 1 4

Program Y prints:

0 0 0 0 1

Marks: 0.5 each (the whole line must match).

Why X: `calls` is static: it exists once and is shared by all active calls, while `mine` is private to each call. The four frames have `mine = 1,2,3,4` and print `4,3,2,1` while unwinding; then `f(3)` returns the final `calls`, 4.

Why Y: every call has its own `calls`, and `calls++` yields the *old* value: `mine` is 0 and `calls` becomes 1. Four frames print 0; `f(3)` returns its own `calls`, 1.

6. (1.5 + 2 = 3.5 marks) Answer the questions below (Only use the answer box to provide answers)

(a) A function M on integers is defined by this rule: if $n > 100$ then $M(n) = n - 10$; otherwise $M(n)$ is the value obtained by applying M to the value of $M(n + 11)$. The program reads n and prints $M(n)$. It terminates for every input asked below.

- (i) Write the code for the blank marked as A.
- (ii) What is printed for the input 1?
- (iii) What is the *largest* input for which the program prints 91?

```
#include <stdio.h>

int M(int n) {
    if (n > 100) return n - 10;
    return _____ [A] _____;
}

int main(void) {
    int n;
    scanf("%d", &n);
    printf("%d\n", M(n));
    return 0;
}
```

(i) A: $M(M(n + 11))$

(ii) Output for input 1: 91

(iii) Input: 101

Marks: 0.5 each.

Hint: Do not expand $M(1)$ downwards (it takes 201 calls). Work upwards from the base case: $M(101) = 91$; $M(100) = M(M(111)) = M(101) = 91$; $M(99) = M(M(110)) = M(100) = 91$; and so on: each $M(n)$ falls back on the previous one. So $M(n) = 91$ for every $n \leq 101$, and $M(n) = n - 10 > 91$ for $n \geq 102$. Hence the largest input giving 91 is 101.

(b) n people, numbered $1, \dots, n$, stand in a circle sequentially. Walking around the circle, the pattern “stays, eliminated, stays, eliminated, ...” is applied: person 1 stays, person 2 is eliminated, person 3 stays, person 4 is eliminated, and so on. The walk keeps going round and round (skipping people already eliminated) with the same alternating pattern, until one person is left. $J(n)$ is that person’s number; for example $J(2) = 1$ and $J(3) = 3$. The program reads n and prints $J(n)$.

- (i) Fill blanks [A] and [B]. (Hint: the values $J(2) = 1$ and $J(3) = 3$ should help.)
- (ii) What is printed for the input 41?
- (iii) What is printed for the input 16389? (Note: $16384 = 2^{14}$.)

```
#include <stdio.h>

int J(int n) {
    if (n == 1) return 1;
    return 2 * J(n / 2)
        + (n % 2 ? ___[A]___ : ___[B]___);
}

int main(void) {
    int n;
    scanf("%d", &n);
    printf("%d\n", J(n));
    return 0;
}
```

(i) Fill blanks:

A: 1

B: -1

(ii) Output: 19

(iii) Output: 11

Marks:

(i) 0.25 + 0.25 (ii) 0.5 (iii) 1

Explanations:

(i) $J(2) = 2J(1) + [F] = 1$ gives $[F] = -1$;

$J(3) = 2J(1) + [E] = 3$ gives $[E] = 1$.

(ii) $41 = 101001_2 \rightarrow 010011_2 = 19$, or evaluate the recursion (6 levels).

(iii) $16389 = 2^{14} + 5$. Writing $n = 2^a + l$ with $0 \leq l < 2^a$ gives $J(n) = 2l + 1$, so $J = 2 \cdot 5 + 1 = 11$. Any correct method earns full marks; the recursion is 15 levels deep, so tracing is possible but slow.

7. ($0.5 \times 3 + 0.5 \times 4 = 3.5$ marks) The following program *measures* how likely it is that a die shows six: it rolls the die N times and divides the number of sixes by N . A computer cannot roll a real die, so `rnd()` makes numbers up by a fixed formula: on every call it takes the stored number s , replaces it by the number $(5s + 3) \bmod 8$ (the remainder after dividing $(5s + 3)$ by 8) and returns it. The numbers look random, but the whole sequence is decided by the starting value 7. **You may use this fact:** in any block of 8 consecutive calls of `rnd()`, each of the values $0, 1, \dots, 7$ appears exactly once, and after 8 calls the stored number is back to 7. `roll()` must return (the remainder of `rnd()` divided by 6) + 1, a number which can vary from 1 to 6.

(a) Fill blanks [S], [Q] and [P] (the fraction of rolls that were sixes, as a real number). Please write your answers in the answer box.

```
#include <stdio.h>

int rnd(void) {
    _____[S]_____ s = 7;
    s = (5 * s + 3) % 8;
    return s;
}

int roll(void) {
    return _____[Q]_____;
}

int main(void) {
    int N, i, sixes = 0;
    double p;
    scanf("%d", &N);
    for (i = 0; i < N; i++)
        if (roll() == 6)
            sixes++;
    p = _____[P]_____;
    printf("%.6f\n", p);
    return 0;
}
```

(a) Fill blanks:

[S]:

`static int (0.5 mark)`
`static (0.25 mark)`

[Q]:

`rnd() % 6 + 1`
Exact match required (0.5 mark)

[P]:

`(double) sixes / N`

Accepted for [P]: any correct real division,
e.g. `(double) sixes / N`, `sixes`
`/ (double) N`, `1.0 * sixes / N`;
plain `sixes / N` earns 0.

(b) If blank [P] is filled by `sixes / N`, what is printed for the input 8?

(c) With everything correct, what is printed for the input 8?

(d) With everything correct, how many sixes are counted for the input 14?

(e) For the input 8000 the correct program prints:

(A) about 0.166667 (B) exactly 0.125000

(C) 0.000000 (D) a different number on every run

(b) Output: 0.000000

(c) Output: 0.125000

(d) Sixes: 2

(e) Choice (A/B/C/D): B

Marks: 0.5 each.

Trace: the stored values are 6, 1, 0, 3, 2, 5, 4, 7, then they repeat. Faces (value mod 6 + 1): 1, 2, 1, 4, 3, 6, 5, 2; exactly one six per block of 8.

(b) 1 six: 1/8 in integer division is 0.

(c) $1/8 = 0.125$.

(d) After 8 draws the state is back at 7, so draws 9–14 repeat draws 1–6, which contain the six: $1 + 1 = 2$.

(e) $8000 = 1000 \times 8$, whole blocks, so exactly 1/8. *More trials cannot cure a biased generator:* 8 values cannot be shared evenly among 6 faces (faces 1 and 2 occur twice per block); the fair value would be 1/6.

8. (6 marks) Answer all questions (answers to be written within the answer boxes only).

(a) (3 marks) What will be printed by the following C program segment?

```
int a = 3, b[] = { 10, 20, 30, 40 };
int *p, *q;
p = &a; q = &b[1];
*(q+1) = *q + *p;
printf("\nOutput: \n");
printf ("\n%d\n", *q+1;
printf ("%d\n", *(q + *p - 2));
printf ("%d\n", b[*p] - *q);
```

Output:

21
23
20

- (b) (3 marks) What will be printed by the following C program?

```
#include <stdio.h>

int *f(int *p, int *q, int r){
    p = q;
    *p = *p + r;
    r = *q + 1;
    q = &r;
    *q = 50;
    return (p);
}

int main(){
    int x = 6, y = 9, z = 3;
    int *w;
    w = f(&x, &y, 5);
    z = *w + x;
    printf("\nOutput: ");
    printf("%d %d %d\n", x, y, z);
    return 0;
}
```

Output:

6 14 20

9. (a) (1.5+1.5+1+1 = 5 marks) A programmer wants to write a program that, given an integer array, produces a new dynamically allocated array holding only the elements that occur more than once in the original array, in order of first occurrence, each appearing exactly once in the output. The programmer does not know the size of the result in advance, so they decide to allocate optimistically and then shrink the block with `realloc` once the true count is known. They also insist on a helper function that returns the result and the count (via pointers), though a C function can return only one value. The program is incomplete. Complete it correctly by filling the blanks. Write your answers in the boxes provided below.

```
#include <stdio.h>
#include <stdlib.h>

/* Returns a pointer to a freshly allocated array of the repeated
values. The number of such values is written back through *m.
Returns NULL if there are none. */

int* repeats(int *a, int n, int *m){
    int i, j, k, seen, count = 0;
    int *r;
    r = _____; /* BLANK 1 */
    if (r == NULL) return NULL;

    for (i = 0; i < n; i++) {
        /* does a[i] occur again anywhere else in a? */
        seen = 0;
        for (j = 0; j < n; j++)
            if (j != i && a[j] == a[i]) seen = 1;
        if (seen == 0) continue;

        /* has a[i] already been recorded in r? */
    }
}
```

```

        for (k = 0; k < count; k++)
            if (r[k] == a[i]) break;
        if (_____) continue;          /* BLANK 2 */
        r[count] = a[i];
        count = count + 1;
    }
    _____;                      /* BLANK 3 */

    if (count == 0) { free(r); return NULL; }
    r = (int *) realloc(r, count * sizeof(int));
    return r;
}

int main(){
    int a[] = {4, 7, 4, 2, 9, 7, 5, 4, 1, 9};
    int n = sizeof(a) / sizeof(int);
    int m, t;    int *res;

    res = repeats(_____);          /* BLANK 4 */

    if (res == NULL)
        printf("none\n");
    else {
        for (t = 0; t < m; t++)
            printf("%d ", res[t]);
        printf("\n%d\n", m);
        free(res);
    }
    return 0;
}

```

BLANK 1: (int*) malloc(n * sizeof(int));

BLANK 2: k < count or count > k (or anything equivalent)

BLANK 3: *m = count

BLANK 4: a, n, &m

- (b) (1 mark) Write the exact output produced by the completed program in part (a)?

Output:

4 7 9
3

10. (3 marks) Consider the following C program, which is intended to dynamically allocate an array of 5 integers, store the values 0, 10, 20, 30, 40 in it, print them, and release the allocated memory before exiting. The required output is: **0 10 20 30 40**

```

#include <stdio.h>
#include <stdlib.h>

```

```

int main(){

```

```

int *p, i;
p = (int *) malloc(5 * sizeof(int));
if (p == NULL)
    { printf("Memory cannot be allocated"); exit(1); }

for (i = 0; i < 5; i++) {
    *p = i * 10;
    p++;
}

for (i = 0; i < 5; i++)    printf("%d ", p[i]);
free(p);
return 0;
}

```

(a) The program contains **one error**. Identify the error and explain it.

Error:

p is advanced by *p++* inside the first loop, so after it finishes *p* points one element past the end of the block. The printing loop then reads *p*[0], ..., *p*[4] from beyond the array (garbage or a crash), and *free(p)* is passed an address that *malloc* never returned.

(b) We now attempt to correct the program in part (a). Complete the program segment below by filling in the two blanks so that it prints **0 10 20 30 40** and releases the allocated memory. You must *not* declare any new variable, and you must *not* change any other line, including *free(p)*;. **Write your answers in the boxes provided below.**

Fill in the two blanks to get a correct program:

```

int *p, i;

p = (int *) malloc(5 * sizeof(int));
if (p==NULL){ printf("Memory cannot be allocated"); exit(1); }

for (i = 0; i < 5; i++) { *p = i * 10; p++; }
_____;    /* BLANK 1 */

for (i = 0; i < 5; i++)
    printf("%d ", _____);    /* BLANK 2 */

free(p);

```

BLANK 1: *p = p-5*

BLANK 2: **(p+i) or p[i]*